

PSE: Einleitung Entwurf



Lukas Hilgert

Lehrstuhl KI für Sprachtechnologien (AI4LT)

Übersicht

- Artefakte
- Inhalt
- Konkrete Schritte
- Bewertung eines Entwurfs
- Anti-Entwurfsmuster

Inhalt von

https://sdq.kastel.kit.edu/wiki/Praxis_der_Software-Entwicklung/Tipps_und_Tricks

Artefakte

- PDF-Dokument mit UML-Diagrammen
- Klassenbeschreibungen
- Erläuterungen
- Entwurfsentscheidungen
- Evtl. großformatiges Klassendiagramm

Inhalt

- Erfahrungsgemäßer **Umfang**:
 - 100 Seiten, primär Klassenbeschreibungen
 - 40–80 Klassen ohne Interfaces
- **Einleitung** mit grobem Überblick.
 - An Pflichtenheft anschließen und Aufteilung in Pakete erklären.
- **Änderungen** zum Pflichtenheft, z.B. gekürzte Wunschkriterien.
- Detaillierte **Beschreibung aller Klassen**
 - Beinhaltet (JavaDoc) Beschreibungen zu allen Methoden, Konstruktoren, Packages und Klassen.
 - Was hier *nicht* reingehört, sind private Felder und Methoden; (Implementierungsdetails)
- Diagramme
 - Generell sollten Diagramme verwendet werden, wenn sich dadurch etwas besser als in Fließtext ausdrücken lässt.
 - Es gibt keine Mindest- oder Höchstanzahl von Diagrammen für das Entwurfsdokument. Die Betreuenden bewerten das Dokument nicht nach Anzahl der Diagramme, sondern danach, ob diese sinnvoll eingesetzt sind.

Inhalt

- Beschreibung von charakteristischen Abläufen anhand von **Sequenz- oder Aktivitätsdiagrammen**.
 - Beispielsweise bieten sich Testszenarien aus dem Pflichtenheft hier an.
 - Wir empfehlen, diese Diagramme möglichst früh zu erstellen, denn dabei werden die Schnittstellen zwischen Packages und Klassen klar.
- Mit Blick auf den Implementierungsplan:
 - Aufteilung in Klassen / Pakete, die unabhängig voneinander implementiert und getestet werden können.
- Vollständiges großformatiges **Klassendiagramm** im Anhang.
 - Ausschnitte / Teile können bereits vorher verwendet werden.
 - Assoziationen zwischen Klassen dabei bitte mit entsprechenden Pfeilen darstellen, statt nur durch Feldtypen.
 - Programmiersprachentypische Funktionen für Attribute (z.B. getter / setter in Java) sollten weggelassen werden, um die Übersicht zu erhöhen.
- Identifikation von **Entwurfsmustern**, um die Struktur gröber zu beschreiben.

Konkrete Schritte

- „Unserer Erfahrung nach ist die Entwurfsphase die schwierigste, da Studierende hier so gut wie keine Erfahrung haben (im Gegensatz zur Implementierungsphase).“
- Sucht alle relevanten Substantive im Pflichtenheft heraus. Viele davon lassen sich direkt als Klassen abbilden. Zum Beispiel: Button, Level, Spielfigur, Bild, etc.
- Was kann man mit diesen Objekten tun? So erhält man die Methoden. Beispielsweise kann man ein Bild drehen, anzeigen und umfärben.
- In welcher Beziehung stehen die Objekte miteinander? So erhält man die Pfeile im UML-Diagramm. Zum Beispiel weiß eine Spielfigur vielleicht, in welchem Level sie ist.
- Spielt die Testfallszenarien durch. So finden sich noch fehlende Methoden und Assoziationen. Außerdem könnt ihr dabei schon die ersten Sequenzdiagramme sammeln.
- Organisiert die Klassen sinnvoll in Gruppen. So erhält man die Paketstruktur. Beispielsweise Model-View-Controller könnte hier sichtbar werden.

Konkrete Schritte

- Diese ersten Schritte kann man erstmal zügig in einer Sitzung mit dem ganzen Team machen. Anschließend kann man die Pakete oder Klassen einzelnen Personen zuordnen, die dann eigenverantwortlich die Details ausarbeiten.
- Nun ist die erste Woche rum. Man hat einiges an Material produziert und kann es dem Betreuer zeigen.
- Jetzt ist es auch an der Zeit mit dem eigentlichen Dokument zu beginnen. Insbesondere sollten technische Fragen geklärt sein. Womit erstellen wir unsere UML-Diagramme? Automatisches Erzeugen von Java-Code mit JavaDoc-Kommentaren und daraus LaTeX und PDF erzeugen? Das alles sollte man in der zweiten Woche zum Laufen bringen.
- Da man nun bereits ein Dokument hat, kann man grundlegende Entwurfsentscheidung sofort niederschreiben. Sammelt erstmal alles in dem Dokument. Ordnen und polieren kann man das später in der Phase.
- Ein weiterer großer Brocken ist die Anbindung an die Plattform, die man benutzt. Beispielsweise Qt, Android oder ein anderes Framework. Hier ist es oft ratsam ein paar minimale Beispielanwendungen zu bauen, um ein Gefühl für die API zu bekommen.

Konkrete Schritte

- Nach zwei Wochen sollten die meisten Klassen entworfen sein. Allerdings hat üblicherweise jedes Teammitglied seinen Bereich für sich bearbeitet. Nun wird es Zeit sich mit der Integration zu beschäftigen. Nehmt euch noch einmal die Testfallszenarien aus dem Pflichtenheft vor und geht diese anhand des Klassendiagramms detailliert durch. Wer ruft wen mit welchen Argumenten auf? Üblicherweise fallen dabei viele Details auf, wo man mehr Informationen weitergeben muss.
- Auch nicht zu vergessen sind Dinge, die nicht im UML auftauchen. Bilder, SQL-Schema, JSON-Schema, Tools wie ein Leveleditor, etc.
- Nun sind drei von vier Wochen rum und eigentlich sollte der Entwurf so ziemlich vollständig sein. Jetzt ist genügend Zeit für Feinschliff, Präsentation und Konsistenzprüfung.

Bewertung eines Entwurfs

- **Geheimnisprinzip** (information hiding) beachtet?
- **Lose Koppelung** zwischen Klassen / Paketen?
- **Keine indirekten Abhängigkeiten?**
- **Starke Kohäsion** innerhalb von Klasse / Paket?
- Klassen / Pakete sind **gleichzeitig erweiterbar** und **stabil**?
- **Liskovsches Substitutionsprinzip** bei Vererbung erfüllt?
- **Verhalten von Implementierung getrennt?**
- **Keine zyklischen Abhängigkeiten?**
- **Lokalitätsprinzip** beachtet?
- Sind Methoden frei von unerwarteten **Nebenwirkungen**?

Anti-Entwurfsmuster

- **God Object**
- **Anemic domain model**
 - Wenn das *Model* praktisch nur noch Datenspeicher ist.
 - Objekt-orientierter Entwurf zeichnet sich gerade dadurch aus, dass Daten *und* Verarbeitung in Objekten zusammengebaut werden.
 - Objekte, die nur zur Datenhaltung da sind, erzwingen prozedurale Programmierung.
- `switch` und `instanceof`

Ausblick: Implementierung

- Artefakt: Implementierungsplan (zu Beginn)
 - Klarer zeitlicher Ablauf der Implementierungsphase, um frühzeitig Verzögerungen zu bemerken.
 - Abhängigkeiten aus dem Entwurf müssen beachtet werden.
 - Wie können Teile der Anwendung möglichst früh getestet werden? Braucht es dafür Stub- / Mock-Klassen?
 - Klare Aufgabenverteilung im Team. Dabei muss eine faire Verteilung und die Abhängigkeiten beachtet werden.
 - Einzelne Jobs kann man Klassen oder Packages zuordnen, aber häufig ist das nicht möglich.
 - Als Form bietet sich ein GANTT Chart an.
 - Zu *Beginn* der Implementierungsphase (d.h. nach 1-2 Tagen) beim Betreuer abzugeben.